

OS-344 Lab Assignment 3: Inter-Process Communication in xv6

Narodia Mit (230123039)
Rehan Sherawat (230123077)
Samir Raj (230123056)
Prakhar Agrawal (230123045)

September 1, 2025

1 Introduction

The xv6 operating system is a teaching kernel that provides a minimal but functional UNIX-like environment. However, its default implementation has very limited support for inter-process communication (IPC). The objective of this assignment was to extend xv6 by introducing two fundamental IPC mechanisms:

- **Shared Memory (shm)**: Efficient data exchange between processes through memory regions mapped into their address spaces.
- **Mailboxes (mbx)**: Message-passing mechanism for reliable synchronization and communication.

These primitives allow xv6 processes to cooperate more effectively and form the foundation for more complex concurrent applications.

2 Kernel Changes for Shared Memory and Mailboxes

2.1 Modifications in main.c

The kernel's boot sequence was extended to initialize new IPC subsystems:

```
1 shm_init();    // initialize shared memory table
2 mbx_init();    // initialize mailbox table
```

This ensures that the shared memory and mailbox structures are ready before user processes are launched. Placement before trap and interrupt initialization guarantees that the IPC primitives are available as soon as processes begin executing.

2.2 System Call Definitions (syscall.h)

New system call numbers were added for shared memory and mailbox operations:

```

1 #define SYS_shm_create 22
2 #define SYS_shm_get    23
3 #define SYS_shm_close  24
4 #define SYS_mbox_create 25
5 #define SYS_mbox_send   26
6 #define SYS_mbox_recv   27

```

This establishes unique identifiers that connect user-space calls to kernel-side handlers.

2.3 System Call Dispatch (syscall.c)

The syscall table was extended with new entries:

```

1 [SYS_shm_create] sys_shm_create,
2 [SYS_shm_get]   sys_shm_get,
3 [SYS_shm_close] sys_shm_close,
4 [SYS_mbox_create] sys_mbox_create,
5 [SYS_mbox_send]  sys_mbox_send,
6 [SYS_mbox_recv]  sys_mbox_recv,

```

This array maps syscall numbers to their corresponding kernel handler functions.

2.4 Function Prototypes (defs.h)

Kernel prototypes for shared memory and mailbox functions were declared:

```

1 // shm.c
2 void  shminit(void);
3 int   shm_create(int);
4 void* shm_get(int);
5 int   shm_close(int);
6
7 // mbox.c
8 void  mboxinit(void);
9 int   mbox_create(int);
10 int   mbox_send(int, int);
11 int   mbox_recv(int, uint64);

```

These declarations ensure that all kernel components can invoke the IPC functions.

2.5 User-Space Stubs (usys.pl and user.h)

Entries were added in `usys.pl` so that assembly stubs for system calls are generated. In `user.h`, user-level function prototypes were defined, allowing applications to call IPC syscalls naturally.

```

1 int shm_create(int key);
2 void* shm_get(int key);
3 int shm_close(int key);
4
5 int mbox_create(int key);
6 int mbox_send(int mbox_id, int msg);
7 int mbox_recv(int mbox_id, int *msg);

```

2.6 System Call Implementations (sysproc.c)

Each system call wrapper extracts user-space arguments and invokes the corresponding kernel routine:

```
1 uint64 sys_shm_create(void) {
2     int key;
3     argint(0, &key);
4     return shm_create(key);
5 }
6
7 uint64 sys_shm_get(void) {
8     int key;
9     argint(0, &key);
10    return (uint64)shm_get(key);
11 }
12
13 uint64 sys_mbox_send(void) {
14     int mbox_id, msg;
15     argint(0, &mbox_id);
16     argint(1, &msg);
17     return mbox_send(mbox_id, msg);
18 }
```

Here, `argint` and `argaddr` ensure safe extraction of syscall arguments from the user stack.

2.7 Virtual Memory Adjustments (vm.c and riscv.h)

A new page table entry flag `PTE_S` was introduced to mark shared pages:

```
1 #define PTE_S (1L << 8) // shared memory flag
```

The `uvmunmap()` function was modified so that pages with this flag are not prematurely freed:

```
1 if(do_free){
2     if((*pte & PTE_S) == 0) {
3         uint64 pa = PTE2PA(*pte);
4         kfree((void*)pa);
5     }
6 }
```

This ensures that shared pages remain allocated until explicitly closed by all processes.

3 Implementation of Shared Memory (shm.c)

The shared memory subsystem was implemented using a global table of regions, each protected by a spinlock. Each region is identified by a unique integer key, mapped to a physical page allocated by `kalloc()`.

3.1 Data Structures

```
1 struct shm_region {
2     int key;
```

```

3  char *p_addr;    // physical page (kernel VA)
4  int ref_count;   // number of processes attached
5 };
6
7 struct {
8     struct spinlock lock;
9     struct shm_region regions[NSHM];
10 } shm_table;

```

This table tracks up to 64 shared memory regions. Each has a reference counter to ensure proper cleanup.

3.2 Initialization

```

1 void shminit(void) {
2     initlock(&shm_table.lock, "shm_table");
3     for (int i = 0; i < NSHM; i++) {
4         shm_table.regions[i].key = 0;
5         shm_table.regions[i].p_addr = 0;
6         shm_table.regions[i].ref_count = 0;
7     }
8     printf("shared memory table initialized\n");
9 }

```

3.3 Creating a Region

```

1 int shm_create(int key) {
2     struct shm_region *r = 0;
3
4     acquire(&shm_table.lock);
5
6     for (int i = 0; i < NSHM; i++) {
7         if (shm_table.regions[i].key == key) {
8             r = &shm_table.regions[i];
9             break;
10        }
11    }
12
13    if (r == 0) {
14        for (int i = 0; i < NSHM; i++) {
15            if (shm_table.regions[i].key == 0) {
16                r = &shm_table.regions[i];
17                r->key = key;
18                r->ref_count = 0;
19                if ((r->p_addr = kalloc()) == 0) {
20                    r->key = 0;
21                    release(&shm_table.lock);
22                    return -1;
23                }
24                memset(r->p_addr, 0, PGSIZE);
25                break;
26            }
27        }
28    }
29 }

```

```

30     release(&shm_table.lock);
31
32     if (r == 0) return -1;
33     return key;
34 }

```

The logic of `shm_create()` is as follows:

- The function first acquires the global lock on the shared memory table to ensure exclusive access.
- It searches the table to check if a region with the requested key already exists.
 - If found, the existing region is reused and returned.
- If no region exists for this key, the function searches for a free slot (where `key == 0`).
 - A new `shm_region` is initialized with the given key, and a page of physical memory is allocated with `kalloc()`.
 - The page is cleared using `memset()` to ensure deterministic behavior for processes attaching to it later.
 - If allocation fails, the function cleans up by resetting the key to 0 and returning `-1`.
- Finally, the lock is released, and either the key of the created/found region is returned or `-1` on failure.

This design ensures that:

1. At most one shared memory region is created for a given key.
2. Access is thread-safe due to spinlock protection.
3. Memory is properly zeroed, preventing leakage of stale data between unrelated processes.

3.4 Attaching to a Region

```

1 void* shm_get(int key) {
2     struct shm_region *r = 0;
3     uint64 va;
4
5     acquire(&shm_table.lock);
6
7     for (int i = 0; i < NSHM; i++) {
8         if (shm_table.regions[i].key == key) {
9             r = &shm_table.regions[i];
10            break;
11        }
12    }
13
14    if (r != 0) {
15        struct proc *curproc = myproc();

```

```

16     va = PGROUNDUP(curproc->sz);
17
18     uint64 pa = (uint64)r->p_addr;
19
20     if (mappages(curproc->pagetable, va, PGSIZE, pa, PTE_R | PTE_W |
21                 PTE_U | PTE_S) < 0) {
22         release(&shm_table.lock);
23         return (void*)-1;
24     }
25
26     sfence_vma(); // flush TLB
27
28     r->ref_count++;
29     curproc->sz = va + PGSIZE;
30 }
31
32 release(&shm_table.lock);
33
34 if (r == 0) return (void*)-1;
35 return (void*)va;
36 }

```

The `shm_get()` function performs the following steps:

- It acquires the global lock on the shared memory table and searches for the region with the requested key.
- If the region exists, it selects a new virtual address in the current process's address space, aligned to the next page boundary (`PGROUNDUP(curproc->sz)`).
- It retrieves the physical page pointer from the region (`r->p_addr`) and maps it into the process's page table using `mappages()` with the flags `PTE_R | PTE_W | PTE_U | PTE_S`.
- The `PTE_S` flag ensures that the page is treated as shared and will not be freed prematurely by `uvmunmap()`.
- After mapping, a TLB flush (`sfence_vma()`) is issued to refresh translation entries.
- The reference count for the region is incremented, and the process's size field (`sz`) is updated to include the new page.

This design guarantees that all processes attaching to the same key end up referencing the same physical memory page, while still seeing it at potentially different virtual addresses in their own address spaces.

3.5 Closing a Region

```

1 int shm_close(int key) {
2     struct shm_region *r = 0;
3
4     acquire(&shm_table.lock);
5
6     for (int i = 0; i < NSHM; i++) {
7         if (shm_table.regions[i].key == key) {

```

```

8     r = &shm_table.regions[i];
9     break;
10 }
11 }
12
13 if (r != 0) {
14     r->ref_count--;
15     if (r->ref_count == 0) {
16         kfree(r->p_addr);
17         r->key = 0;
18         r->p_addr = 0;
19     }
20 }
21
22 release(&shm_table.lock);
23
24 if (r == 0) return -1;
25 return 0;
26 }

```

The `shm_close()` function is responsible for detaching a process from a shared memory region:

- It acquires the lock on the shared memory table and searches for the region matching the provided key.
- If found, the region's reference counter is decremented to indicate that one fewer process is using it.
- If the counter reaches zero, the shared memory page is freed using `kfree()`, and the region's entry is reset by clearing its key and address.
- The lock is released before returning, ensuring proper synchronization across processes.

This approach guarantees that shared memory pages are freed only when the last process detaches, preventing premature deallocation while maintaining safe resource cleanup.

3.6 Test Program (`shmtest.c`)

A user-level test was written to demonstrate shared memory usage between parent and child processes. The parent creates a region, writes to it, and forks a child. The child attaches to the same region and observes the parent's data.

```

1 #include "kernel/types.h"
2 #include "kernel/stat.h"
3 #include "user/user.h"
4
5 int main(void) {
6     int key = 1234;
7     int *shared_mem;
8
9
10    printf("shmtest: starting\n");
11
12    if (shm_create(key) < 0) {

```

```

13     fprintf(2, "shmtest: shm_create failed\n");
14
15     exit(1);
16 }
17
18 shared_mem = shm_get(key);
19 if (shared_mem == (int*)-1) {
20     fprintf(2, "shmtest: shm_get failed\n");
21     exit(1);
22 }
23
24 printf("shmtest: parent mapped shared memory at %p\n", shared_mem);
25
26 *shared_mem = 50;
27 printf("shmtest: parent wrote 50 to shared memory\n");
28
29 int pid = fork();
30
31 if (pid < 0) {
32     fprintf(2, "shmtest: fork failed\n");
33     exit(1);
34 } else if (pid == 0) {
35     // Child process
36     int *child_mem;
37
38     child_mem = shm_get(key);
39     if (child_mem == (int*)-1) {
40         fprintf(2, "shmtest: child shm_get failed\n");
41         exit(1);
42     }
43
44     printf("shmtest: child mapped shared memory at %p\n", child_mem);
45     printf("shmtest: child reads %d from shared memory\n", *
46         child_mem);
47
48     *child_mem = 100;
49     printf("shmtest: child wrote 100 to shared memory\n");
50
51     if (shm_close(key) < 0) {
52         fprintf(2, "shmtest: child shm_close failed\n");
53     }
54
55     exit(0);
56 } else {
57     // Parent process
58     wait(0);
59
60     printf("shmtest: parent reads %d from shared memory after child
61         modification\n", *shared_mem);
62
63     if (shm_close(key) < 0) {
64         fprintf(2, "shmtest: parent shm_close failed\n");
65     }
66 }
67
68 printf("shmtest: finished\n");

```



```
68     exit(0);
69
70     return 0;
71 }
72 }
```

The test program follows these steps:

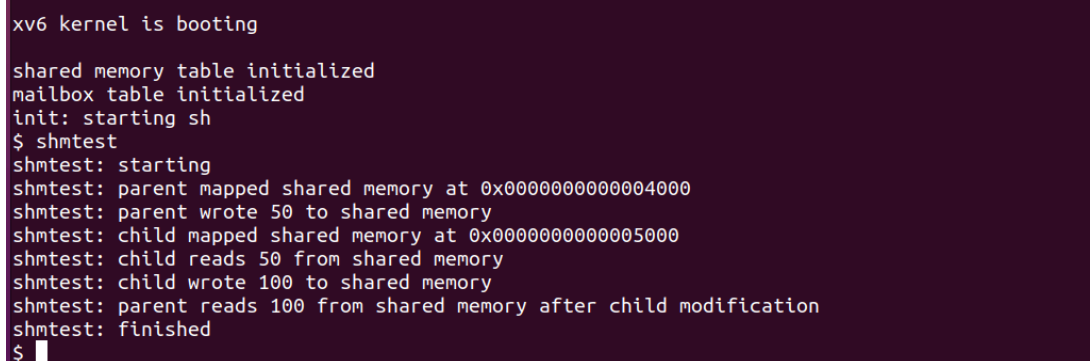
1. The parent process creates a shared memory region using `shm_create()` and attaches to it with `shm_get()`.
2. It writes the integer value 50 into the shared memory to initialize it.
3. The process then calls `fork()`:
 - The child process attaches to the same region with `shm_get()`, confirming that it sees the value written by the parent.
 - The child writes a new value 100 into the shared memory and closes the region with `shm_close()`.
 - The child then terminates.
4. Meanwhile, the parent waits for the child to finish. After the child exits, the parent re-reads the value in shared memory and confirms it has been updated to 100.
5. Finally, the parent calls `shm_close()` and exits.

This program demonstrates two key aspects:

- Shared memory allows both parent and child processes to access the same physical page, even though they have different virtual addresses.
- Updates made by one process are immediately visible to the other, proving correct mapping and synchronization.

3.7 Observed Output

When executed, the test produced the following output:



```
xv6 kernel is booting
shared memory table initialized
mailbox table initialized
init: starting sh
$ shmtest
shmtest: starting
shmtest: parent mapped shared memory at 0x0000000000004000
shmtest: parent wrote 50 to shared memory
shmtest: child mapped shared memory at 0x0000000000005000
shmtest: child reads 50 from shared memory
shmtest: child wrote 100 to shared memory
shmtest: parent reads 100 from shared memory after child modification
shmtest: finished
$
```

Figure 1: Successful execution of the `shmtest` program.

3.8 Detailed Justification of Observed Output

The printed messages from `shmtest` align with the expected behavior:

- **Kernel boot messages:** “xv6 kernel is booting”, “shared memory table initialized”, and “mailbox table initialized” confirm that the system started properly and our IPC subsystems were initialized at boot.
- **Parent mapping:** The line showing the parent mapped the page at `0x4000` indicates that a free virtual page was allocated and correctly mapped to the new shared memory region.
- **Parent write:** The parent stores the value 50, initializing the shared region.
- **Child mapping and read:** The child maps the same physical page (but at a different virtual address `0x5000`) and reads the value 50, proving that both processes see the same underlying memory.
- **Child write:** The child updates the value to 100, which is immediately visible to any process attached to this page.
- **Parent re-read:** The parent confirms the update by reading 100, demonstrating successful bidirectional sharing of data.
- **Completion:** The final “shmtest: finished” shows that both processes closed their regions and exited cleanly without errors.

This sequence validates that the shared memory implementation provides consistent, synchronized access across processes, fulfilling the intended IPC functionality.

4 Implementation of Mailboxes (`mbox.c`)

The mailbox subsystem was designed to provide message-based inter-process communication in xv6. Unlike shared memory, mailboxes allow processes to exchange discrete integer messages, with built-in blocking semantics to synchronize senders and receivers.

4.1 Data Structures

```
1 struct mailbox {
2     int key;                // Identifier for this mailbox
3     int messages[MBOX_SIZE]; // Circular buffer for messages
4     int head;               // Read index
5     int tail;               // Write index
6     int count;              // Number of messages currently stored
7     int in_use;             // Active flag
8 };
9
10 struct {
11     struct spinlock lock;
12     struct mailbox mailboxes[NMAILBOX];
13 } mbox_table;
```

This design supports up to 64 mailboxes, each storing 3 integer messages. The circular buffer with head/tail pointers ensures efficient enqueue/dequeue operations.

4.2 Initialization

```
1 void mboxinit(void) {
2     initlock(&mbox_table.lock, "mbox_table");
3     acquire(&mbox_table.lock);
4     for (int i = 0; i < NMAILBOX; i++) {
5         mbox_table.mailboxes[i].in_use = 0;
6         mbox_table.mailboxes[i].head = 0;
7         mbox_table.mailboxes[i].tail = 0;
8         mbox_table.mailboxes[i].count = 0;
9     }
10    release(&mbox_table.lock);
11    printf("mailbox table initialized\n");
12 }
```

4.3 Creating a Mailbox

```
1 int mbox_create(int key) {
2     struct mailbox *mb = 0;
3
4     acquire(&mbox_table.lock);
5
6     // Check if a mailbox with this key already exists.
7     for (int i = 0; i < NMAILBOX; i++) {
8         if (mbox_table.mailboxes[i].in_use && mbox_table.mailboxes[i].key
9             == key) {
10            mb = &mbox_table.mailboxes[i];
11            break;
12        }
13    }
14
15    // If not found, create a new one in an empty slot.
16    if (mb == 0) {
17        for (int i = 0; i < NMAILBOX; i++) {
18            if (mbox_table.mailboxes[i].in_use == 0) {
19                mb = &mbox_table.mailboxes[i];
20                mb->key = key;
21                mb->in_use = 1;
22                mb->head = 0;
23                mb->tail = 0;
24                mb->count = 0;
25                break;
26            }
27        }
28    }
29
30    release(&mbox_table.lock);
31
32    if (mb == 0) return -1; // Table is full
33    return key;
34 }
```

The function performs the following steps:

- It first acquires the global lock to ensure mutual exclusion when accessing the mailbox table.

- The function checks if a mailbox with the given key already exists:
 - If found, the existing mailbox is returned, allowing multiple processes to share the same mailbox by key.
- If no such mailbox exists, the function searches for a free slot (an unused mailbox entry).
 - On finding one, it initializes the mailbox with the provided key, resets its head/tail pointers, clears the message count, and marks it active (`in_use = 1`).
- The lock is then released, and the function returns the key of the created or reused mailbox.
- If no free slot was available, the function returns `-1`, signaling that the mailbox table is full.

This design guarantees that:

1. Each mailbox is uniquely identified by a key.
2. Multiple processes can attach to the same mailbox using the same key.
3. Table integrity is preserved via spinlock protection, preventing race conditions during mailbox creation.

4.4 Sending a Message

```

1 int mbox_send(int mbox_id, int msg) {
2     struct mailbox *mb = 0;
3
4     acquire(&mbox_table.lock);
5
6     // Find the mailbox.
7     for (int i = 0; i < NMAILBOX; i++) {
8         if (mbox_table.mailboxes[i].in_use && mbox_table.mailboxes[i].key
9             == mbox_id) {
10             mb = &mbox_table.mailboxes[i];
11             break;
12         }
13     }
14
15     if (mb == 0) {
16         release(&mbox_table.lock);
17         return -1; // Mailbox not found
18     }
19
20     // Critical section: Wait if the mailbox is full.
21     while (mb->count == MBOX_SIZE) {
22         // Atomically release lock and sleep, will re-acquire lock on
23         // wakeup.
24         sleep(mb, &mbox_table.lock);
25     }
26 }
```

```

25 // Add the message to the circular queue.
26 mb->messages[mb->tail] = msg;
27 mb->tail = (mb->tail + 1) % MBOX_SIZE;
28 mb->count++;
29
30 // Wake up any processes that might be sleeping, waiting to receive.
31 wakeup(mb);
32
33 release(&mbox_table.lock);
34 return 0;
35 }

```

The function works as follows:

- The global lock is acquired to ensure safe concurrent access to the mailbox table.
- It searches the table for the mailbox with the matching key (`mbox_id`). If none is found, it returns `-1`.
- If the mailbox is found but already at full capacity (`count == MBOX_SIZE`), the sender process is put to sleep using `sleep()`, releasing the lock atomically until space becomes available.
- Once space is free, the new message is placed in the circular buffer at `tail`, and the tail pointer and count are updated accordingly.
- The function then calls `wakeup()` to notify any waiting receiver processes that new data is available.
- Finally, the lock is released, and the function returns `0` on success.

This design ensures that:

1. Sending is safe in a concurrent environment through spinlock protection.
2. The mailbox implements proper blocking semantics: a sender will wait if the mailbox is full, preventing data loss.
3. Backpressure is applied to the sender, ensuring balanced communication between producer and consumer processes.

4.5 Receiving a Message

```

1 int mbox_rcv(int mbox_id, uint64 msg_addr) {
2     struct mailbox *mb = 0;
3     int received_msg;
4
5     acquire(&mbox_table.lock);
6
7     // Find the mailbox.
8     for (int i = 0; i < NMAILBOX; i++) {
9         if (mbox_table.mailboxes[i].in_use && mbox_table.mailboxes[i].key
10             == mbox_id) {
11             mb = &mbox_table.mailboxes[i];
12             break;
13         }
14     }
15     if (!mb) return -1;
16
17     // ... (receiving logic) ...
18 }

```

```

12     }
13 }
14
15 if (mb == 0) {
16     release(&mbox_table.lock);
17     return -1; // Mailbox not found
18 }
19
20 // Critical section: Wait if the mailbox is empty.
21 while (mb->count == 0) {
22     sleep(mb, &mbox_table.lock);
23 }
24
25 // Retrieve the message from the circular queue.
26 received_msg = mb->messages[mb->head];
27 mb->head = (mb->head + 1) % MBOX_SIZE;
28 mb->count--;
29
30 // Safely copy the message from kernel space to the user-space
   address.
31 if(copyout(myproc()->pagetable, msg_addr, (char*)&received_msg,
   sizeof(int)) < 0) {
32     release(&mbox_table.lock);
33     return -1;
34 }
35
36 // Wake up any processes that might be sleeping, waiting to send.
37 wakeup(mb);
38
39 release(&mbox_table.lock);
40 return 0;
41 }

```

The function works as follows:

- The global lock is acquired to ensure exclusive access to the mailbox table.
- It searches the table for a mailbox with the given key (`mbox_id`). If no such mailbox exists, the function returns `-1`.
- If the mailbox exists but is currently empty (`count == 0`), the receiver process is put to sleep using `sleep()`, releasing the lock atomically until a sender provides new data.
- Once a message is available, it is removed from the circular buffer at the `head` index. The head pointer and count are updated to reflect the removal.
- The message is then copied safely from kernel space into the user-provided buffer using `copyout()`.
- After retrieving the message, `wakeup()` is called to notify any blocked senders that space is now available.
- Finally, the lock is released, and the function returns `0` on success.

This design ensures that:

1. Receiving is safe in a concurrent environment through spinlock protection.
2. The mailbox provides proper blocking semantics: a receiver waits if the mailbox is empty, ensuring no spurious reads.
3. Communication remains synchronized, as senders and receivers are awakened exactly when progress is possible.

4.6 Test Program (mboxtest.c)

Two phases of testing were implemented:

1. **Blocking on Empty Mailbox:** A child process attempts to receive from an empty mailbox, blocking until the parent sends a message. The child then resumes and reads the sent value.
2. **Blocking on Full Mailbox:** The parent fills the mailbox to capacity. When attempting to send another message, it blocks until the child consumes one. Once the child receives, the sender is unblocked and continues.

```

1 #include "kernel/types.h"
2 #include "kernel/stat.h"
3 #include "user/user.h"
4
5 // Test that a process correctly blocks when receiving from an empty
   mailbox.
6 void test_empty_block()
7 {
8     int key = 2001;
9     int msg_to_send = 101;
10    int received_msg;
11
12    printf("\n--- Phase 1: Testing block on EMPTY mailbox ---\n");
13
14    if(mbox_create(key) < 0) {
15        fprintf(2, "Phase 1: mbox_create failed\n");
16        return;
17    }
18
19    int pid = fork();
20    if(pid < 0) {
21        fprintf(2, "Phase 1: fork failed\n");
22        return;
23    }
24
25    if(pid == 0){
26        // Child (Receiver)
27        printf("Receiver: Attempting to receive from empty mailbox (
           should block)...\n");
28        mbox_rcv(key, &received_msg);
29        printf("Receiver: Unblocked and received message: %d\n",
           received_msg);
30        exit(0);
31    } else {
32        // Parent (Sender)

```

```

33
34         // Ensure child runs first and blocks
35     pause(5);
36     printf("Sender: Sending message %d to wake up receiver...\n",
37           msg_to_send);
38     mbox_send(key, msg_to_send);
39     wait(0); // Now we wait for the child to finish completely.
40     printf("Sender: Child finished.\n");
41 }
42
43 // Test that a process correctly blocks when sending to a full mailbox.
44 void test_full_block()
45 {
46     int key = 2002;
47     // Note: MBOX_SIZE must be 3 in kernel/mbox.c for this to work as
48     // intended
49     const int MBOX_CAPACITY = 3;
50
51     printf("\n--- Phase 2: Testing block on FULL mailbox ---\n");
52
53     if(mbox_create(key) < 0) {
54         fprintf(2, "Phase 2: mbox_create failed\n");
55         return;
56     }
57
58     int pid = fork();
59     if(pid < 0) {
60         fprintf(2, "Phase 2: fork failed\n");
61         return;
62     }
63
64     if(pid == 0){
65         // Child (Receiver)
66         int msg;
67         // Sleep to let parent fill mailbox first
68         pause(5);
69         for(int i = 0; i < MBOX_CAPACITY + 1; i++){
70             printf("Receiver: Attempting to receive message #%d...\n",
71                   i+1);
72             mbox_recv(key, &msg);
73             printf("Receiver: Received message %d\n", msg);
74         }
75         exit(0);
76     } else {
77         // Parent (Sender)
78         printf("Sender: Filling mailbox with capacity %d...\n",
79               MBOX_CAPACITY);
80         for(int i = 0; i < MBOX_CAPACITY; i++){
81             mbox_send(key, i + 1);
82             printf("Sender: Sent message %d\n", i + 1);
83         }
84
85         printf("Sender: Mailbox is now full.\n");
86         printf("Sender: Attempting to send 4th message (should block)
87               ...\n");
88         mbox_send(key, MBOX_CAPACITY + 1); // This call should block
89         printf("Sender: Unblocked! 4th message sent successfully.\n");

```



```

86
87     wait(0); // Wait for child to finish
88     printf("Sender: Child finished.\n");
89 }
90 }
91
92 int main(void) {
93     test_empty_block();
94     test_full_block();
95     exit(0);
96     return 0;
97 }

```

4.7 Explanation of Test Program (mbxtest.c)

The test program demonstrates the mailbox system through two scenarios, implemented using parent-child processes created via `fork()`:

- **Phase 1 (Empty Mailbox):** The child blocks on `mbx.recv()` until the parent sends a message, which wakes it up to complete reception.
- **Phase 2 (Full Mailbox):** The parent fills the mailbox; its fourth send blocks until the child consumes one message. Once space frees, the sender resumes, and the child drains the mailbox.

4.8 Observed Output

The execution produced:

```

xv6 kernel is booting

shared memory table initialized
mailbox table initialized
init: starting sh
$ mbxtest

--- Phase 1: Testing block on EMPTY mailbox ---
Receiver: Attempting to receive from empty mailbox (should block)...
Sender: Sending message 101 to wake up receiver...
Receiver: Unblocked and received message: 101
Sender: Child finished.

--- Phase 2: Testing block on FULL mailbox ---
Sender: Filling mailbox with capacity 3...
Sender: Sent message 1
Sender: Sent message 2
Sender: Sent message 3
Sender: Mailbox is now full.
Sender: Attempting to send 4th message (should block)...
Receiver: Attempting to receive message #1...
Receiver: Received message 1
Receiver: Attempting to receive message #2...
Receiver: Received message 2
Receiver: Attempting to receive message #3...
Receiver: Received message 3
Receiver: Attempting to receive message #4...
Sender: Unblocked! 4th message sent successfully.
Receiver: Received message 4
Sender: Child finished.
$

```

Figure 2: Successful execution of the `mbxtest` program.

4.9 Explanation of mboxtest Output

The output of the test program confirms the intended behavior of mailboxes:

- In **Phase 1**, the receiver process prints that it is attempting to read from an empty mailbox and blocks. Only after the sender posts the message does the receiver wake up, print the received value, and continue. The parent then reports that the child has finished, demonstrating that blocking and wakeup are functioning correctly.
- In **Phase 2**, the sender fills the mailbox with three messages (equal to its capacity). On attempting to send the fourth message, it prints that it should block. At this point, the receiver consumes a message, freeing up a slot. Immediately, the blocked sender is unblocked and sends the fourth message successfully. The receiver then continues to read the remaining messages until completion, and finally the parent reports that the child has finished.

Overall, the output validates that:

1. Processes attempting to receive from an empty mailbox block until a sender provides data.
2. Processes attempting to send to a full mailbox block until a receiver consumes data.
3. Synchronization between sender and receiver is correctly handled through sleep-wakeup mechanisms inside the kernel.

5 Task 2: Coordinated Traversal using Shared Memory and Mailboxes

In this task, we extend the previous primitives (`shm` and `mbox`) to implement a coordinated traversal problem. The idea is to have two processes (`Process 1` and `Process 2`) move along an intertwined path stored in shared memory. At each step, one process informs the other of its current location through mailboxes, and the other uses that information to look up its own next location in the shared memory. This problem highlights the combined use of shared memory for data exchange and mailboxes for synchronization.

5.1 Design Rationale

The design involves three main components:

- **Shared Memory:** Used to store a path structure accessible by both processes. Each entry in the shared memory maps one process's current location to the other process's next location. This acts as the shared "map" that guides traversal.
- **Mailboxes:** Serve as synchronization primitives. Each process must know the partner's current location before it can decide its next step. Mailboxes guarantee blocking semantics: if one process has not yet produced its location, the other waits.
- **Deadlock Avoidance:** If both processes tried to send first, they would block indefinitely waiting for space in the mailbox. If both tried to receive first, they would block waiting for input that never arrives. To prevent this, the communication order is made asymmetric:

- Process 1: **send** first, then **receive**.
- Process 2: **receive** first, then **send**.

This asymmetry ensures progress and prevents circular wait conditions.

5.2 Master Program (`master.c`)

The `master` program prepares the environment for the worker processes. Its responsibilities are:

1. Create a shared memory page using `shm_create()`.
2. Create two mailboxes, one for messages from Process 1 to Process 2, and one for messages in the reverse direction.
3. Initialize the shared memory to encode the intertwined traversal path. For example:
 - Index 0 (P1's start) points to 10, meaning P2's next location is 10.
 - Index 1 (P2's start) points to 20, meaning P1's next location is 20.
 - This continues until the special marker `END_MARKER` indicates the path terminates.
4. Fork twice, executing the worker program `process` with the correct arguments: process ID, start index, shared memory key, and send/receive mailbox keys.
5. Wait for both processes to complete and clean up the shared memory region.

```

1 #include "kernel/types.h"
2 #include "kernel/stat.h"
3 #include "user/user.h"
4
5 // Define constants for IPC keys
6 #define SHM_KEY 1000
7 #define MBOX_P1_TO_P2_KEY 2001
8 #define MBOX_P2_TO_P1_KEY 2002
9
10 // Define path structure
11 #define P1_START_IDX 0
12 #define P2_START_IDX 1
13 #define END_MARKER 0xFFFF
14
15 int main(void) {
16     int *shm_page;
17
18     printf("master: starting...\n");
19
20     // 1. Create the shared memory page.
21     if (shm_create(SHM_KEY) < 0) {
22         fprintf(2, "master: shm_create failed\n");
23         exit(1);
24     }
25
26     // 2. Create the two mailboxes.
27     if (mbox_create(MBOX_P1_TO_P2_KEY) < 0 || mbox_create(
        MBOX_P2_TO_P1_KEY) < 0) {

```

```

28     fprintf(2, "master: mbox_create failed\n");
29     exit(1);
30 }
31
32 // 3. Attach to shared memory and initialize the intertwined path.
33 shm_page = shm_get(SHM_KEY);
34 if (shm_page == (int*)-1) {
35     fprintf(2, "master: shm_get failed\n");
36     exit(1);
37 }
38
39 printf("master: initializing shared memory path...\n");
40 // The value at P1's location (index 0) is the next location for P2
41 // The value at P2's location (index 1) is the next location for P1
42
43 shm_page[P1_START_IDX] = 10;    // P2's next location is 10
44 shm_page[P2_START_IDX] = 20;    // P1's next location is 20
45 shm_page[10] = 30;              // P1's next location is 30
46 shm_page[20] = 40;              // P2's next location is 40
47 shm_page[30] = END_MARKER;      // P2's path ends
48 shm_page[40] = END_MARKER;      // P1's path ends
49 printf("master: initialization complete.\n");
50
51 // 4. Fork and exec the two worker processes.
52 // We fork twice from the master to create two concurrent children.
53
54 if (fork() == 0) {
55     // This is the first child, which will become Process 1.
56     // Arguments for exec: process_id, start_idx, shm_key, send_key
57     // , recv_key
58     char *argv[] = {"process", "1", "0", "1000", "2001", "2002",
59                     0};
60     exec("process", argv);
61     fprintf(2, "master: exec process 1 failed\n");
62     exit(1);
63 }
64
65 if (fork() == 0) {
66     // This is the second child, which will become Process 2.
67     // Note the swapped send/recv keys.
68     char *argv[] = {"process", "2", "1", "1000", "2002", "2001",
69                     0};
70     exec("process", argv);
71     fprintf(2, "master: exec process 2 failed\n");
72     exit(1);
73 }
74
75 // 5. Wait for both child processes to finish.
76 printf("master: waiting for processes to finish...\n");
77 wait(0);
78 wait(0);
79
80 // 6. Clean up the shared memory region (optional but good practice
81 // ).
82 shm_close(SHM_KEY);
83
84 printf("master: finished.\n");

```

```

80     exit(0);
81     return 0;
82 }

```

5.3 Worker Program (process.c)

Each worker process attaches to the shared memory and begins traversal. Its responsibilities are:

1. Attach to the shared memory using `shm_get()`.
2. Based on its ID, decide its communication protocol:
 - Process 1: sends its location to Process 2, then waits to receive P2's location.
 - Process 2: receives P1's location first, then sends its own.
3. Once the partner's location is received, use it as an index into shared memory to compute the process's next location.
4. Continue this loop until the `END_MARKER` is encountered, which signals that traversal has finished.
5. Detach from shared memory and terminate.

```

1  #include "kernel/types.h"
2  #include "kernel/stat.h"
3  #include "user/user.h"
4
5  #define END_MARKER 0xFFFF
6
7  int main(int argc, char *argv[]) {
8      if (argc < 6) {
9          fprintf(2, "process: usage: process id start_idx shm_key\n");
10         send_key rcv_key\n");
11         exit(1);
12     }
13
14     int my_id = atoi(argv[1]);
15     int my_location = atoi(argv[2]);
16     int shm_key = atoi(argv[3]);
17     int send_key = atoi(argv[4]);
18     int rcv_key = atoi(argv[5]);
19
20     int *shm_page;
21     int partner_location;
22     int my_next_location = my_location; // Initialize to start location
23
24     // Attach to the shared memory region.
25     shm_page = shm_get(shm_key);
26     if (shm_page == (int*)-1) {
27         fprintf(2, "process %d: shm_get failed\n", my_id);
28         exit(1);
29     }

```

```

30     printf("Process %d: starting at location %d.\n", my_id, my_location
31           );
32     // Main traversal loop.
33     while (my_next_location != END_MARKER) {
34
35         // Implement the deadlock avoidance strategy.
36         if (my_id == 1) {
37             // Process 1: Send first, then Receive
38             mbox_send(send_key, my_location);
39             mbox_rcv(rcv_key, &partner_location);
40         } else {
41             // Process 2: Receive first, then Send
42             mbox_rcv(rcv_key, &partner_location);
43             mbox_send(send_key, my_location);
44         }
45
46         // Use the location received to look up our own next location.
47         my_next_location = shm_page[partner_location];
48
49         // Update local position.
50         my_location = my_next_location;
51
52         // Print progress for the report.
53         if (my_location != END_MARKER) {
54             printf("Process %d: received partner's location, my new
55                   location is %d\n", my_id, my_location);
56         }
57     }
58     printf("Process %d: finished traversal.\n", my_id);
59
60     // Detach from shared memory before exiting.
61     shm_close(shm_key);
62
63     exit(0);
64     return 0;
65 }

```

5.4 Observed Output

The execution produced:

Step-by-Step Explanation

- **Initialization:** The master program creates a shared memory page and two mailboxes. It then encodes the intertwined traversal path in shared memory:
 - At index 0 (P1's start), the entry is 10 → meaning P2's next location will be 10.
 - At index 1 (P2's start), the entry is 20 → meaning P1's next location will be 20.
 - At index 10, the entry is 30, and at index 20, the entry is 40.
 - Both 30 and 40 point to `END_MARKER`, terminating the traversal.

```

xv6 kernel is booting

shared memory table initialized
mailbox table initialized
init: starting sh
$ master
master: starting...
master: initializing shared memory path...
master: initialization complete.
master: waiting for processes to finish...
Process 1: starting at location 0.
Process 2: starting at location 1.
Process 2: received partner's location, my new location is 10
Process 1: received partner's location, my new location is 20
Process 2: received partner's location, my new location is 40
Process 1: received partner's location, my new location is 30
Process 2: finished traversal.
Process 1: finished traversal.
master: finished.
$

```

Figure 3: Successful execution of the `master` program.

This setup is confirmed by the lines:

```

master:  initializing shared memory path...
master:  initialization complete.

```

- **Launching Processes:** The master forks two children and executes `process.c`. They attach to the shared memory and begin traversal:

```

Process 1:  starting at location 0.
Process 2:  starting at location 1.

```

- **Traversal and Synchronization:**

1. Process 1 sends its current location (0) to Process 2.
2. Process 2 receives this, looks up shared memory at index 0, and finds its own next location = 10.
3. Process 2 then sends its location (1) to Process 1.
4. Process 1 receives this, looks up index 1 in shared memory, and updates its location to 20.

This matches the logs:

```

Process 2:  received partner's location, my new location is 10
Process 1:  received partner's location, my new location is 20

```

- **Continuing the Path:** Using the same logic:

- Process 2 uses partner's latest location (20) to find its new location = 40.
- Process 1 uses partner's latest location (10) to find its new location = 30.

Confirmed by:

```

Process 2:  received partner's location, my new location is 40
Process 1:  received partner's location, my new location is 30

```

- **Termination:** At locations 30 and 40, shared memory contains `END_MARKER`, so both processes terminate their traversal. The log shows:

```
Process 2: finished traversal.  
Process 1: finished traversal.
```

Finally, the master process waits for both children and finishes:

```
master: finished.
```

Justification of Output

The output confirms that:

1. **Shared Memory Works:** Both processes are able to read the correct next locations from the shared memory path structure.
2. **Mailbox Synchronization Works:** Each process blocks appropriately until it receives its partner's location. The asymmetric send/receive ordering prevents deadlocks.
3. **Traversal is Correct:** The intertwined path encoded by `master.c` is followed step-by-step until both processes reach `END_MARKER`.
4. **System Completeness:** The master sets up the environment, the worker processes carry out traversal, and all resources are cleaned up correctly at the end.

This demonstrates that the integration of shared memory and mailboxes achieves safe, synchronized, and deadlock-free coordination between concurrent processes.

References

- [1] Russ Cox, Frans Kaashoek, and Robert Morris. *Xv6, a simple, Unix-like teaching operating system*.
- [2] YouTube. (2018). *System Call Implementation in xv6*. Retrieved from <https://www.youtube.com/watch?v=QtZ7PtedIic>.
- [3] Mythili Vutukuru. *CS 347: Operating Systems, IIT Bombay*. Retrieved from <https://www.cse.iitb.ac.in/~mythili/os/>.